

©2025 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

Robust Perception-Based Navigation using PAC-NMPC with a Learned Value Function

Adam Polevoy^{1,2}, Mark Gonzales^{1,2}, Marin Kobilarov², Joseph Moore^{1,2}

Abstract—Nonlinear model predictive control (NMPC) is typically restricted to short, finite horizons to limit the computational burden of online optimization. As a result, global planning frameworks are frequently necessary to avoid local minima when using NMPC for navigation in complex environments. By contrast, reinforcement learning (RL) can generate policies that minimize the expected cost over an infinite-horizon and can often avoid local minima, even when operating only on current sensor measurements. However, these learned policies are usually unable to provide performance guarantees (e.g., on collision avoidance), especially when outside of the training distribution. In this paper, we augment Probably Approximately Correct NMPC (PAC-NMPC), a sampling-based stochastic NMPC algorithm capable of providing statistical guarantees of performance and safety, with an approximate perception-based value function trained via RL. We demonstrate in simulation that our algorithm can improve the long-term behavior of PAC-NMPC while outperforming other approaches with regards to safety for both planar car dynamics and more complex, high-dimensional fixed-wing aerial vehicle dynamics. We also demonstrate that, even when our value function is trained in simulation, our algorithm can successfully achieve statistically safe navigation on hardware using a 1/10th scale rally car in cluttered real-world environments using only current sensor information.

I. INTRODUCTION

Nonlinear model predictive control (NMPC) has been an effective approach for perception-based robot navigation (e.g., [1], [2], [3], [4]). In some cases, robust and stochastic NMPC (RNMP and SNMPC respectively) can provide performance guarantees in the presence of uncertainty [5], [6], [7]. However, NMPC is typically restricted to short, finite horizons to limit the computational burden of online optimization. This often limits controller performance, since the cost accrued over the short horizon is often a poor approximation of the true cost-to-go. In navigation tasks, this limitation can cause the system to get caught in local minima. This is especially true in cluttered environments and in “reactive” control paradigms that only leverage current sensor measurements. Moreover, hand-designing a terminal cost that incorporates perception data and effectively approximates the true cost-to-go can be prohibitively difficult.

A common approach to improve the global performance of a receding-horizon controller is to utilize a global planner to generate receding-horizon waypoints. These global planners, however, often do not utilize the true system dynamics,

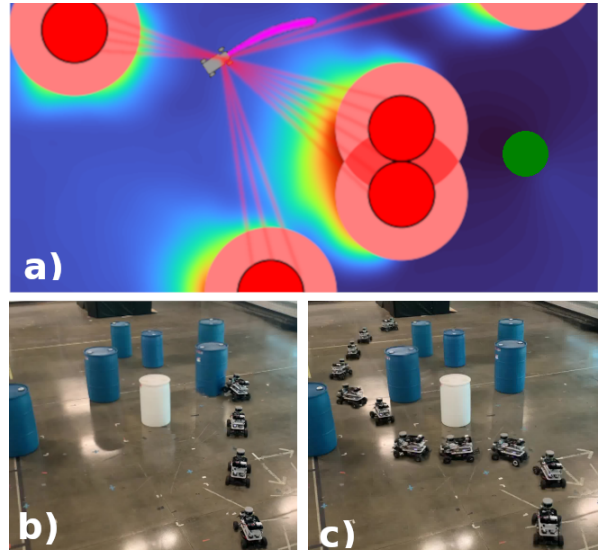


Fig. 1. a) Rally car navigating through obstacle field using NMPC with learned value function and LiDAR. b) Timelapse of rally car navigating through barrels using RL trained policy. c) Timelapse of rally car navigating through barrels using PAC-NMPC with learned value function.

which can be high-dimensional for complex systems. Rather, they tend to leverage simplified approximations with reduced dimensionality to improve computational tractability (e.g., [8], [3], [9]). There are also challenges associated with implementing an effective global planner for perception-based navigation, such as planning for potential hidden obstacles in obscured areas. In these cases, the performance of the overall system can be limited by the global planner.

By contrast, reinforcement learning (RL) can train effective control policies which directly use current perception information to avoid local minima that can be observed or inferred in complex environments [10], [11], [12]. However, because these RL algorithms require large amounts of data, they are often trained in simulation (e.g., [13], [14], [15]). This dependence on simulation and the inability of the RL trained policies to guarantee the satisfaction of general state constraints can frequently lead to degraded, and potentially unsafe, performance when the policy is deployed in the real-world or when it is used in out-of-distribution environments.

In this paper, we augment Probably Approximately Correct NMPC (PAC-NMPC) [7] with an RL-trained perception-based value function to achieve navigation with statistical performance guarantees even when the learned value function is approximate and uncertain. Using Monte Carlo dropout to capture network uncertainty, we learn a stochastic model of the value function. We then use PAC-NMPC

¹Johns Hopkins University Applied Physics Lab
{Adam.Polevoy, Mark.Gonzales,
Joseph.Moore}@jhuapl.edu

²Johns Hopkins University Whiting School of Engineering
mkobilar1@jhu.edu

to minimize a bound on the expected terminal costs and constraints derived from this value function. This encourages our receding-horizon NMPC algorithm to assume the long-range behavior of the RL policy, while preserving the statistical safety guarantees afforded by minimizing the sample-complexity bounds derived in [16]. We demonstrate through simulation and hardware experiments that our algorithm can achieve probabilistically safe perception-based navigation, improved sim-to-real transfer, robustness to out-of-distribution scenarios, and scales to complex, nonlinear high-dimensional dynamic systems.

II. RELATED WORK

To enable effective finite horizon NMPC, researchers have investigated the use of learned models to generate waypoints directly from sensor data [17]. Learned waypoints have been used in MPC for drone racing [18], quadrotor navigation in environments with dead-end corridors [19], navigation in real-world cluttered environments [20], and navigation in dynamic environments with other agents [21]. Recently, learned waypoints have been used alongside artificial potential fields [22] and hierarchical reinforcement learning [23] to improve navigation in complex environments when using only immediate sensor information.

By contrast, our approach aims to use reinforcement learning to augment the cost function of NMPC directly, which is more closely related to Quasi-Infinite Horizon NMPC [24]. A more recent approach suggested the use of a learned approximate value function [25] to determine the terminal cost. This idea was built upon to update the approximate value function online [26], learn an approximate dynamics model [27], formulate the running cost as an importance sampler of the value function [28], and to weigh the value function against local Q-function approximations along the trajectories [29].

Safe RL methods aim to generate a policy that can minimize a cost while satisfying safety constraints [30], [31]. Often these problems are posed as Constrained Markov Decision Processes (CMDP) and are solved using Lagrangian methods [32], [33], or via constrained policy gradient methods [34]. Other methods, such as Conservative Safety Critic [35], learn a safety critic, which is used to select safe actions to execute. Safety Editor Policy [36] satisfies constraints by learning an additional policy that maps unsafe actions to safe actions. Control barrier functions (CBF) (e.g., [37]), Lyapunov functions (e.g., [38]), and backwards reachable sets (e.g., [39]) are also used for ensuring safety in terms of RL constraint satisfaction. RL-CBF [40] learns a Gaussian Process model for the unknown dynamics to generate a CBF, which is then used as a safety filter on the RL-policy and to improve exploration efficiency. However, this approach assumes the algorithm is provided with a valid safe set, which can be very challenging to compute. Another approach, [41], adds a CBF to the value function and proves that the learned actor and critic will converge to a safe, optimal solution. However, the CBF must be known and included in the augmented reward function. Some of these

model-based safe RL approaches have been demonstrated in hardware (e.g., CBF methods for autonomous driving in [42] and reachable set methods for legged locomotion [43]). Similar to our method, Actor Critic Props [44], uses sample complexity bounds to provide probabilistic guarantees on the performance and safety of learned policies. Generally, the guarantees produced by this method do not hold outside of the training distribution. Safe RL has also been formulated using robust MPC, rather than neural networks, as the function approximator [45], [46]. Although this inherently generates policies that satisfy constraints, it is limited by the parameterization of the MPC policy.

To the best of our knowledge, our approach is the first to combine a learned perception-based stochastic value function with NMPC to achieve probabilistic guarantees for navigation and obstacle avoidance.

III. BACKGROUND

A. PAC-NMPC

Probably Approximately Correct NMPC (PAC-NMPC) [7] is a sampling-based SNMPC method which minimizes an upper confidence bound on the expected cost and probability of constraint violation of a local feedback policy.

Consider the stochastic dynamical system given by the probability density function $p(\mathbf{x}_{t+1}|\mathbf{x}_t, \mathbf{u}_t)$, where $\mathbf{x}_t \in \mathbb{R}^{N_x}$ is a vector of state values and $\mathbf{u}_t \in \mathbb{R}^{N_u}$ is a vector of control inputs. PAC-NMPC uses Iterative Stochastic Policy Optimization (ISPO) [16] to formulate the search for a control policy, $\mathbf{u}_t = \pi(\mathbf{x}_t, \xi)$, as a stochastic optimization problem. This is achieved by sampling the policy parameters, ξ , from a surrogate distribution given by the probability density function $p(\xi|\nu)$ and defined by hyper-parameters ν .

More specifically, PAC-NMPC optimizes a time-varying local feedback policy of the form $\mathbf{u}_t = \mathbf{K}_t(\tau^d(\xi, \mathbf{x}_0))(\mathbf{x}_t^d(\xi, \mathbf{x}_0) - \mathbf{x}_t) + \mathbf{u}_t^d(\xi)$ where $\tau^d \triangleq \{\mathbf{x}_0^d, \mathbf{u}_0^d, \mathbf{x}_1^d, \mathbf{u}_1^d, \dots, \mathbf{u}_{N_T-1}^d, \mathbf{x}_{N_T}^d\}$ is the nominal trajectory computed using a nominal deterministic discrete-time dynamics model $\mathbf{x}_{t+1}^d = \mathbf{x}_t^d + \mathbf{f}(\mathbf{x}_t^d, \mathbf{u}_t^d)\Delta t$ over N_T time steps. Here $\mathbf{x}_t^d \in \mathbb{R}^{N_x}$ and $\mathbf{u}_t^d \in \mathbb{R}^{N_u}$ are the nominal states and control inputs at time index t respectively. Δt is the discrete time step and $\mathbf{K}_t(\tau^d(\xi, \mathbf{x}_0)) \in \mathbb{R}^{N_u \times N_x}$ is a sequence of time-varying feedback gains computed using the finite horizon, discrete, time-varying linear quadratic regulator (TVLQR). The policy is parameterized only by the nominal input sequence, so that $\xi = [\mathbf{u}_0^{dT}, \mathbf{u}_1^{dT}, \dots, \mathbf{u}_{N_T-1}^{dT}]^T$. The surrogate distribution, $p(\xi|\nu)$, is parameterized as a multivariate Gaussian, $\mathcal{N}(\xi|\mu, \Sigma)$, with a mean, μ , and a covariance matrix, Σ . Since the covariance matrix is diagonal, the hyperparameters can be written as $\nu \triangleq [\mu^T \text{diag}(\Sigma)^T]^T$.

For a discrete-time trajectory sequence $\tau = \{\mathbf{x}_0, \mathbf{u}_0, \mathbf{x}_1, \mathbf{u}_1, \dots, \mathbf{u}_{N_T-1}, \mathbf{x}_{N_T}\}$, one can define a non-negative trajectory cost function, $J(\tau) \geq 0$, and a trajectory constraint violation function $C(\tau) \in \{0, 1\}$. During each planning interval, PAC-NMPC iteratively optimizes $\nu^* = \arg \min_{\nu} \min_{\alpha > 0} (\mathcal{J}_{\alpha}^+(\nu) + \gamma \mathcal{C}_{\alpha}^+(\nu))$, where $\mathcal{J}_{\alpha}^+(\nu)$

is the PAC bound on $J(\tau)$, $\mathcal{C}_\alpha^+(\nu)$ is the PAC bound on $C(\tau)$, and $\gamma > 0$ is a heuristically selected weighting coefficient. $\mathcal{J}_\alpha^+(\nu)$ and $\mathcal{C}_\alpha^+(\nu)$ take the form

$$\begin{aligned}\mathcal{J}_\alpha^+(\nu) &\triangleq \hat{\mathcal{J}}_\alpha(\nu) + \alpha d(\nu) + \Phi_\alpha(\delta), \\ \hat{\mathcal{J}}_\alpha(\nu) &\triangleq \frac{1}{\alpha LM} \sum_{i=0}^{L-1} \sum_{j=1}^M \zeta(\alpha \ell_{ij}), \\ \zeta(x) &= \log \left(1 + x + \frac{1}{2} x^2 \right), \ell_{ij} = J(\tau_{ij}) \frac{p(\xi_{ij}|\nu)}{p(\xi_{ij}|\nu_i)}, \\ d(\nu) &\triangleq \frac{1}{2L} \sum_{i=0}^{L-1} b_i^2 e^{D_2(p(\cdot|\nu)||p(\cdot|\nu_i))}, \Phi_\alpha(\delta) \triangleq \frac{1}{\alpha LM} \log \frac{1}{\delta}, \\ 0 &\leq J(\tau_{ij}) \leq b_i \quad \forall j = 0, \dots, M\end{aligned}\tag{1}$$

where $\hat{\mathcal{J}}_\alpha(\nu)$ is a robust estimator of the expected cost, $d(\nu)$ is a distance between distributions, and $\Phi_\alpha(\delta)$ is a concentration-of-measure term, $\alpha > 0$ is an annealing coefficient, $1 - \delta$ is the bound confidence, L is the number of prior policies, M is the number of samples per policy, and D_2 is the Renyi divergence.

These PAC bounds were derived in [16] and guarantee that $\mathbb{P}(\mathbb{E}_{\tau, \xi \sim p(\cdot|\nu)}[J(\tau)] \leq \mathcal{J}_\alpha^+(\nu)) \geq 1 - \delta$ and $\mathbb{P}(\mathbb{E}_{\tau, \xi \sim p(\cdot|\nu)}[C(\tau)] \leq \mathcal{C}_\alpha^+(\nu)) \geq 1 - \delta$. They are not only optimization targets, but also provide probabilistic guarantees of performance and safety. For instance, setting $\delta = 0.05$ means that with 95% confidence the expected cost will not exceed $\mathcal{J}_\alpha^+$, while the probability of constrained violation will not exceed $\mathcal{C}_\alpha^+$.

B. Actor Critic Reinforcement Learning

Actor Critic algorithms are policy gradient approaches to solve Markov decision processes (MDP) by simultaneously learning a policy and action-value function [47]. Consider a MDP with states $\mathbf{s}_t \in \mathbb{R}^{N_s}$, actions $\mathbf{a}_t \in \mathbb{R}^{N_a}$, transition dynamics $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, and reward function $r(\mathbf{s}_t, \mathbf{a}_t)$, where t denotes the time index. The discounted return is defined as the sum of the discounted future rewards of a state-action sequence, $R_t^{\gamma_d} = \sum_{i=t}^{\infty} \{\gamma_d^{i-t} r(\mathbf{s}_i, \mathbf{a}_i)\}$ where, $\gamma_d \in [0, 1)$ is a discount factor. The action-value function (Q-function), $Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}[R_t^{\gamma_d}|\mathbf{s}_t, \mathbf{a}_t]$, is the expected discounted return after taking an action $\mathbf{a}_t$ from state $\mathbf{s}_t$ and then following policy $\mathbf{a}_t = \pi(\mathbf{s}_t)$. The value function, $V^\pi(\mathbf{s}_t) = Q^\pi(\mathbf{s}_t, \pi(\mathbf{s}_t))$, is the expected discounted return of following a policy, $\mathbf{a}_t = \pi(\mathbf{s}_t)$, from a given state $\mathbf{s}_t$. Actor-critic algorithms learn parameters ϕ for an actor policy, $\pi^\phi(\mathbf{s}_t)$, and parameters ψ for a critic action-value function, $Q^\psi(\mathbf{s}_t, \mathbf{a}_t)$, which estimate the optimal policy $\pi^*(\mathbf{s}_t)$ and action-value function $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t)$ where $\pi^*(\mathbf{s}_t) = \mathbf{a}_t^* = \arg \min_{\mathbf{a}_t} \mathbb{E}[Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t)]$ and $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t^*) = \mathbb{E}[r(\mathbf{s}_t, \mathbf{a}_t^*) + \gamma_d Q^{\pi^*}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}^*)]$.

IV. APPROACH

A. Problem Formulation

Our objective is for a robot to navigate through an obstacle field from an initial state, $\mathbf{x}_I$, to a goal state, $\mathbf{x}_G$. The robot is equipped with LiDAR which returns range measurements

$\ell_t = [\ell_t^0 \ \ell_t^1 \ \dots \ \ell_t^{N_\ell}]$ at bearings $\beta = [\beta^0 \ \beta^1 \ \dots \ \beta^{N_\ell}]$ with a maximum range of ℓ_{max} at each timestep. We formulate a generic cost on a finite horizon trajectory,

$$J(\tau) = \sum_{i=0}^{N_T-1} \{q(\mathbf{x}_i, \mathbf{u}_i)\} + q_f(\mathbf{x}_{N_T}), \tag{2}$$

where $q(\mathbf{x}_i, \mathbf{u}_i)$ is the running cost and $q_f(\mathbf{x}_{N_T})$ is the terminal cost. We also formulate constraints to bound the state and prevent collisions with obstacles:

$$\begin{aligned}g_b(\mathbf{x}_t) &= (\mathbf{x}_t - \mathbf{x}_l < 0) \vee (\mathbf{x}_u - \mathbf{x}_t < 0), \\ g_o(\mathbf{x}_t) &= \neg((dist(\mathbf{x}_t, \mathbf{p}_{oj}) > r) \vee j), \\ c(\mathbf{x}_t) &= g_b(\mathbf{x}_t) \vee g_o(\mathbf{x}_t), \\ C(\tau) &= c(\mathbf{x}_0) \vee c(\mathbf{x}_1) \vee \dots \vee c(\mathbf{x}_{N_T}).\end{aligned}\tag{3}$$

Here, $\vee$ and $\neg$ are the logical or and not operators, respectively. $\mathbf{x}_l$ is the lower state bound, $\mathbf{x}_u$ is the upper state bound, r is the maximum robot radius. $\mathbf{p}_{oj}$ is an observed occupied point in the world frame, calculated as

$$\mathbf{p}_{oj} = \mathbf{p}_t + [\cos(\theta_t + \beta^j) \quad \sin(\theta_t + \beta^j)]^T \ell_t^j \tag{4}$$

for each $\ell_t^j < \ell_{max}$ where $\mathbf{p}_t$ and θ_t are the position and orientation of the robot at state $\mathbf{x}_t$.

B. Learned Value Function

Using our probabilistic dynamics model, sensor model, costs and constraints, we define the components of the general MDP referenced in Section III-B. Let $\mathbf{s}_t = \mathbf{h}(\mathbf{x}_t, \ell_t)$, $\mathbf{a}_t = \mathbf{u}_t$, and

$$r(\mathbf{s}_t, \mathbf{a}_t) = -q(\mathbf{x}_t, \mathbf{u}_t) - \gamma_r c(\mathbf{x}_t), \tag{5}$$

where $\mathbf{h}(\mathbf{x}_t, \ell_t)$ maps the robot state and sensor measurements to the MDP state representation and γ_r is a heuristically selected constraint violation penalty. Although this problem is actually a partially observable MDP (POMDP), we approximate it as an MDP by including the LiDAR measurements as a part of the state vector, which is a common approach [48], [49], [50].

We learn an actor policy, $\pi^\phi(\mathbf{s}_t)$, and a critic action-value function, $Q^\psi(\mathbf{s}_t, \mathbf{a}_t)$, using the CleanRL implementation [51] of TD3 [52], a state-of-the-art actor-critic method. However, our approach is agnostic to the method used to train the actor and critic networks. The value function is then reconstructed as $V^{\psi\phi}(\mathbf{x}_t, \ell_t) = Q^\psi(\mathbf{s}_t, \pi^\phi(\mathbf{s}_t))$.

C. PAC-NMPC with Learned Value Function

To use the learned value function as a terminal state cost, we must estimate the future LiDAR measurement, $\hat{\ell}_{N_T}$, at the terminal state based on the current LiDAR measurement, ℓ_0 . The performance guarantees on the value function assume that this future sensor prediction is accurate. We calculate the range, $\hat{\ell}^j$, and bearing, $\hat{\beta}^j$, to each observed occupied point from the terminal state of the trajectory as $\hat{\ell}^j = \|\mathbf{p}_{N_T} - \mathbf{p}_{oj}\|$, and $\hat{\beta}^j = \text{atan2}(\mathbf{p}_{N_T} - \mathbf{p}_{oj}) - \theta_{N_T}$. Since the bearing estimates do not necessarily correspond to the discrete bearing values used by the sensor, we assign the the estimated LiDAR measurements to the closest bearing $\hat{\ell}_{N_T}^k = \hat{\ell}^j$ where $k = \arg \min_k \{|\hat{\beta}^j - \beta^k|\}$.

We use the negative learned value function as the terminal cost so that $q_f(\mathbf{x}_{N_T}) = -V^{\psi\phi}(\mathbf{x}_{N_T}, \hat{\ell}_{N_T})$. In this way, we approximate the infinite horizon trajectory cost given the current LiDAR measurement. We apply the additional constraint that the learned value function improves from the current state to the final state of the trajectory:

$$\begin{aligned} g_V(\mathbf{x}_{N_T}) &= V(\mathbf{x}_{N_T}, \hat{\ell}_{N_T}) - V(\mathbf{x}_0, \ell_0) \\ c_V(\mathbf{x}_{N_T}) &= g_V(\mathbf{x}_{N_T}) < 0 \\ C(\tau) &= c(\mathbf{x}_0) \vee c(\mathbf{x}_1) \cdots \vee c(\mathbf{x}_{N_T}) \vee c_V(\mathbf{x}_{N_T}). \end{aligned} \quad (6)$$

During each planning interval, PAC-NMPC optimizes and returns policy hyperparameters, ν^* , a PAC bound on the expected cost, $\mathcal{J}_\alpha^+(\nu^*)$, and a PAC bound on the probability of constraint violation, $\mathcal{C}_\alpha^+(\nu^*)$. These bounds serve as performance and safety guarantees for the policy; the probability of violating the obstacle constraint or worsening the learned value function, given the current LiDAR measurement, will be less than $\mathcal{C}_\alpha^+(\nu)$ with a probability of $1 - \delta$ (eq. III-A).

V. SIMULATION EXPERIMENTS

A. Experimental Setup

We simulate a stochastic bicycle model with acceleration and steering rate inputs given as

$$\begin{aligned} \mathbf{x}_{t+1} &\sim p(\cdot | \mathbf{x}_t, \mathbf{u}_t) \triangleq \mathbf{x}_t + (f(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\omega}) \Delta t, \\ f(\mathbf{x}_t, \mathbf{u}_t) &= [v \cos(\theta), v \sin(\theta), v \tan(\delta_s)/l, \dot{v}, \dot{\delta}_s]^T, \\ \boldsymbol{\omega} &\sim \mathcal{N}(\cdot | \mathbf{0}, \boldsymbol{\Gamma}). \end{aligned} \quad (7)$$

Here $\mathbf{x}_t = [p_x \ p_y \ \theta \ v \ \delta_s]^T$ is the state vector, $\mathbf{u}_t = [\dot{v} \ \dot{\delta}_s]^T$ is the control vector, $l = 0.33m$ is the wheelbase, $\boldsymbol{\omega}$ is Gaussian noise, and $\boldsymbol{\Gamma} = \text{diag}([4e^{-4}, 4e^{-4}, 1.1e^{-2}, 1e^{-1}, 5.6e^{-3}])$ is the process covariance, which was fit from hardware data. We limit the acceleration, $\dot{v} \in [-1., 1] \frac{m}{s}$, the steering rate, $\dot{\delta}_s \in [-1, 1] \frac{rad}{s}$, and the steering angle, $\delta_s \in [-0.4, 0.4] rad$.

We use a quadratic state cost $q(\mathbf{x}, \mathbf{u}) = (\mathbf{x} - \mathbf{x}_G)^T Q (\mathbf{x} - \mathbf{x}_G)$ where $Q = \text{diag}([1e^{-2} \ 1e^{-2} \ 0 \ 0 \ 0])$, and a velocity constraint such that $v \in [-1, 3] \frac{m}{s}$.

Our approach considers not only uncertainty in the dynamics, but also in the learned actor and critic networks. For every trajectory sampled by PAC-NMPC, we also sample dropout masks for the actor and critic networks [53] before evaluating the cost-to-go at the trajectory's terminal state. Since our approach already requires multiple forward passes through the networks, one for each trajectory, Monte Carlo dropout does not add additional inference time. This allows us to incorporate value function uncertainty into the PAC bound to provide more accurate performance guarantees.

We compared our approach against four baselines across two simulation environment distributions. The actor and critic networks were trained for each environment. First, we evaluated PAC-NMPC with a quadratic terminal cost, $q_f(\mathbf{x}_{N_T}) = (\mathbf{x}_{N_T} - \mathbf{x}_G)^T Q_f (\mathbf{x}_{N_T} - \mathbf{x}_G)$ where $Q_f = \text{diag}([1 \ 1 \ 0 \ 0 \ 0])$. Second, we used the LiDAR measurements to continuously build and maintain an occupancy grid of the environment. Each planning iteration, we ran a naive A* search algorithm on the grid to find the shortest path to the

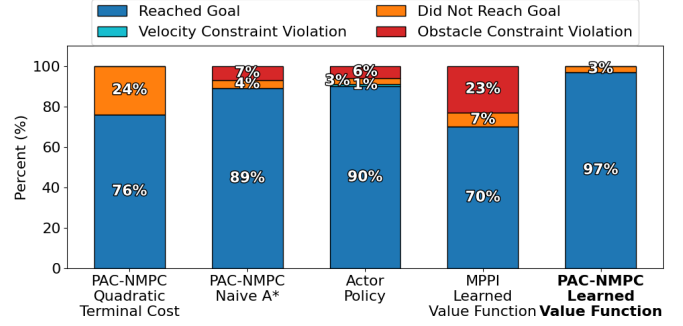


Fig. 2. Simulated cluttered environments. Percentage of trials in which the system reached the goal, didn't reach the goal, and violated constraints.

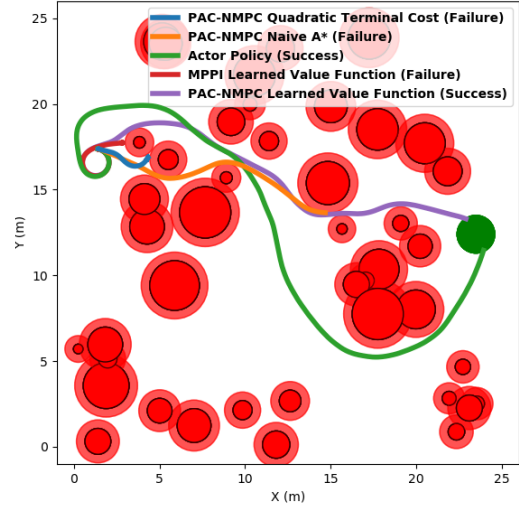


Fig. 3. Simulated cluttered environments. Testing environment where PAC-NMPC fails when only using a quadratic terminal cost.

goal. A receding horizon goal, $\mathbf{x}_A$, was selected at distance of $v_{max} \cdot N_t \cdot \Delta t$ along the path, and was used to form a quadratic terminal cost, $q_f(\mathbf{x}_{N_T}) = (\mathbf{x}_{N_T} - \mathbf{x}_A)^T Q_f (\mathbf{x}_{N_T} - \mathbf{x}_A)$. Third, we compared directly against the actor policy, $\pi^\phi(s)$. Fourth, instead of using PAC-NMPC, we optimized trajectories with MPPI [54], while still using the learned value function as a terminal trajectory cost.

When running PAC-NMPC, we optimize a 12 timestep trajectory with $\Delta t = 0.1$ sec at a replanning period of $H = 0.2$ sec, and we interpolate the feedback policies to 50Hz. We set PAC-NMPC parameters as $L = 5$, $M = 1024$, $\delta = 0.05$, and normalize the trajectory costs before optimization to achieve tighter PAC bounds. We evaluated PAC-NMPC with several γ values and selected the best value for each environment; $\gamma = 2$ in the cluttered environments and $\gamma = 4$ in the concave trap environments. When running MPPI, we similarly normalized the sampled trajectory costs and summed them with the constraints using the same γ values. We used the same number of timesteps, Δt , H , number of sampled trajectories, a temperature of $\gamma_t = 0.35$, and a sampling variance of $\Sigma_\epsilon = 0.01$. We allowed MPPI to run for as many iterations as possible in the replanning period and interpolated the resulting trajectories to 50Hz.

We trained the actor and critic in simulation with the

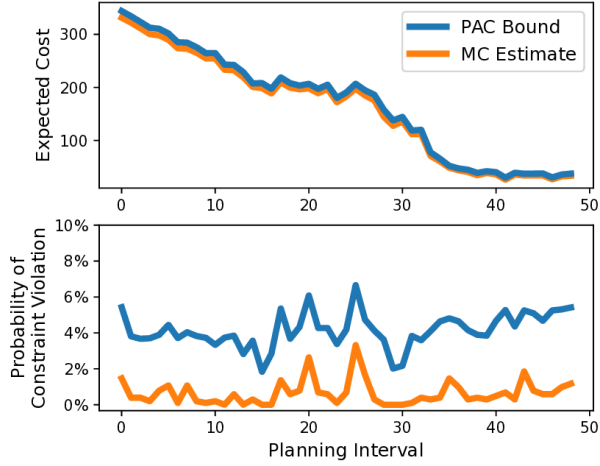


Fig. 4. Optimized PAC Bounds compared to Monte Carlo estimates at each planning interval for example trial when using the learned value function.

stochastic bicycle model and a 64 beam 360° LiDAR. The MDP state, s_t , consists of the normalized velocity, normalized tangent of the steering angle, normalized range to the goal, cosine and sine of the bearing to the goal, and normalized LiDAR measurement. The reward directly mirrors the NMPC costs and constraints (Eq. 5) with $\gamma_r = 1000$. We used shallow, fully connected neural networks as the function approximators for $\pi^\phi(s_t)$ and $Q^\psi(s_t, a_t)$. Simulations were run on a laptop with an Intel Core i9-13900H CPU and a Nvidia GeForce RTX 4080 Max-Q GPU.

B. Cluttered Environments

We generated 100 random environments from the training distribution. Environments consisted of an initial state, x_I , a goal state x_G , and circular obstacles. The obstacles were allowed to overlap, which allowed the constraint regions to combine to form complex environments. To provide meaningful testing environments, environments in which no obstacles blocked the path to the goal were discarded.

Results are shown in Figure 2, with an example environment in Figure 3. When using a quadratic terminal cost, the system often got caught in local minima. When using A*, the system occasionally planned paths through obscured obstacles, which caused constraint violations if the LiDAR was unable to view the obscured obstacles until too close to recover. The actor policy was unable to explicitly enforce constraints which resulted in violations. Our approach outperformed all baselines and never violated the constraints.

In Figure 4, we plot the optimized PAC bounds at each planning interval for an example trial when using the learned value function as a terminal cost and constraint. To validate PAC bounds, we compare them against Monte Carlo estimates of the expected cost and probability of constraint violation, which were formed by sampling 1024 trajectories and dropout masks. On average, PAC-NMPC produced guarantees that the probability of constraint violation of the NMPC generated policies would be less than 5%.

To demonstrate that our approach incorporates uncertainty from the actor and critic networks into the PAC bound com-

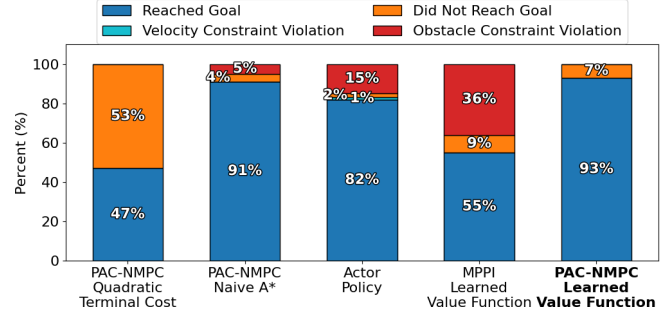


Fig. 5. Simulated concave trap environments. Percentage of concave trap trials in which the system reached the goal, didn't reach the goal, and violated constraints.

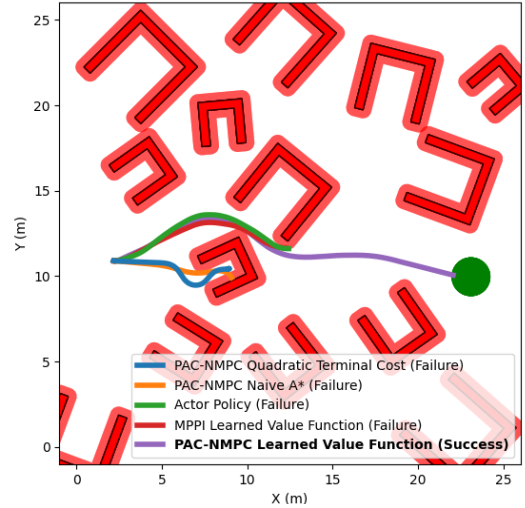


Fig. 6. Simulated concave trap environments. Concave trap environment where the actor policy fails and PAC-NMPC fails with both the quadratic terminal cost and with A*.

putation, we compared the percentage of bound violations that occurred when optimizing the bounds with and without sampling dropout masks. In both cases, we compare against Monte Carlo estimates using 1024 sampled trajectories and dropout masks. We found that across all trials, when optimizing with sampled dropout masks, the expected cost bound was never violated, and the probability of constraint violation bound was violated in only 0.34% of planning intervals. When optimizing without sampled dropout masks, the expected cost bound was violated in 65.49% of planning intervals, and the probability of constraint violation bound was violated in 1.45%.

C. Concave Trap Environments

We also evaluated our approach in environments constructed solely of concave traps to highlight its ability to safely avoid local minima. We uniformly sampled the number of obstacles between 0 and 20, the side lengths of each obstacle between 2.5 and 5.0 meters, and the obstacles were not allowed to overlap. The angles of the obstacles were sampled uniformly such that they were pointing towards the starting position of the robot $\pm \frac{\pi}{2}$ radians. An example environment is shown in Figure 6. We generated 100 random

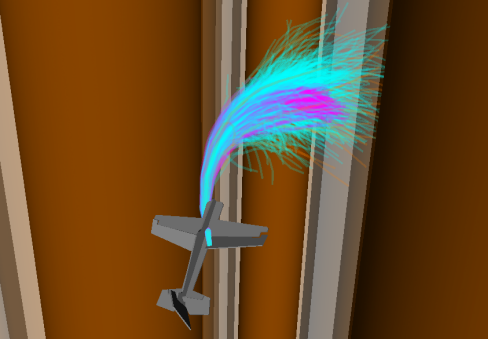


Fig. 7. Fixed-wing simulation environment.

environments from the training distribution to evaluate our approach. Our approach outperformed all baselines and never violated the constraints (Fig. 5)

D. Fixed-wing UAV

We repeat our approach with stochastic fixed-wing UAV dynamics to demonstrate that it can extend to high-dimensional systems (Fig. 7). We utilize the formulation of the fixed-wing described in [55] with 2nd order Runge-Kutta (RK2) integration. The state of the system is $\mathbf{x} = [\mathbf{r}, \mathbf{q}, \boldsymbol{\delta}, \mathbf{v}, \boldsymbol{\omega}, p]$ where $\mathbf{r} \in \mathbb{R}^3$ is the position, $\mathbf{q} \in \mathbb{Q}$ is the orientation, $\boldsymbol{\delta} \in \mathbb{R}^3$ are control surface deflections, $\mathbf{v} \in \mathbb{R}^3$ is the linear velocity, $\boldsymbol{\omega} \in \mathbb{R}^3$ is the angular velocity, and p is the propeller speed. The control signal is the rate of change of the control surface deflections and of the propeller speed: $\mathbf{u} = [u_a, u_e, u_r, u_p]$. We place Gaussian noise over the body frame accelerations, with means of $[0.10, -0.90, -0.89] \frac{m}{s^2}$, $[1.18, -0.16, 1.71] \frac{rad}{s^2}$ and variances of $diag([1.71, 1.63, 1.41])$, $diag([10.05, 5.57, 15.23])$, which was fit from data collected on hardware. We simulated the fixed-wing with a 64 beam 360° LiDAR attached on a gimbal such that it remains parallel to the xy-plane.

We use a quadratic state cost $q(\mathbf{x}, \mathbf{u}) = (\mathbf{r} - \mathbf{x}_G)^T Q_r (\mathbf{r} - \mathbf{x}_G) + \boldsymbol{\delta}^T Q_\delta \boldsymbol{\delta} + \boldsymbol{\omega}^T Q_\omega \boldsymbol{\omega}$ where $Q_r = Q_\omega = diag([0.01, 0.01, 0.01])$ and $Q_\delta = diag([0.1, 0.1, 0.1])$. We constrained the altitude to $[0, 5]m$, the speed along the x, y plane to $\leq 8 \frac{m}{s}$, and the roll rate to $[-10, 10] \frac{rad}{s}$. We set $L = 1$, $\gamma = 3$, and sampled cluttered environments with a maximum of 20 obstacles. The actor and critic networks trained in 3 million steps over 4.5 hours. In these experiments, we do not count velocity constraint violations in Figures 8, 9. Our approach outperformed both the actor policy and PAC-NMPC with a quadratic terminal cost of $q_f(\mathbf{x}, \mathbf{u}) = 10.0 \cdot q(\mathbf{x}, \mathbf{u})$ (Fig. 8).

To demonstrate that a value function trained on simple dynamics can aid in controlling more complex systems, we ran experiments utilizing the bicycle value function to control the fixed-wing. We trained the bicycle value function in these environments with $v_{min} = 2$, $v_{max} = 8$, and no Gaussian noise. Then, we mapped the fixed-wing state to the bicycle state by extracting p_x, p_y, θ, v from $\mathbf{r}, \mathbf{q}, \mathbf{v}$ and setting $\delta = atan(\frac{L}{v}\dot{\theta})$. For the terminal cost, we applied the bicycle value function plus $q(\mathbf{x}, \mathbf{u})$ where $Q_r = diag([0.00, 0.00, 0.1])$,

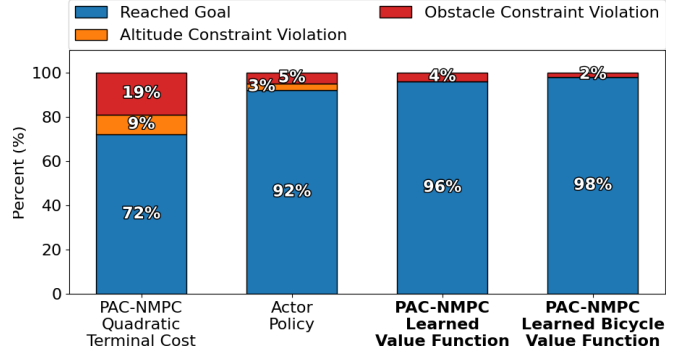


Fig. 8. Fixed-wing environments. Percentage of trials in which the system reached the goal or violated constraints.

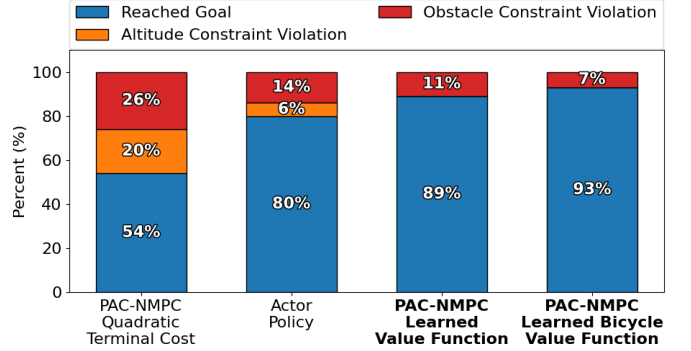


Fig. 9. Fixed-wing environments with additional noise. Percentage of trials in which the system reached the goal or violated constraints.

$Q_\omega = diag([0.1, 0.1, 0.1])$ and $Q_\delta = diag([1, 1, 1])$. This outperformed all other approaches, including PAC-NMPC with the fixed-wing value function (Fig. 8). We hypothesize that this may be because the bicycle value function is easier to learn due to its smaller state space.

To demonstrate that our approach is more resilient when the dynamics uncertainty does not match the training environment, we shifted the mean of the noise over the body accelerations to $[3.10, 2.10, 2.11] \frac{m}{s^2}$ and $[4.18, 2.84, 4.71] \frac{rad}{s^2}$ and repeated the experiments. Our approach was able to maintain higher success rates with both the fixed-wing value function and the bicycle value function than PAC-NMPC with a quadratic terminal cost or the actor policy (Fig. 9).

VI. HARDWARE EXPERIMENTS

To evaluate our approach on physical hardware, we used a 1/10th scale Traxxas Rally Car platform with a Velodyne Puck LITE LiDAR. We modeled the rally car dynamics as a stochastic bicycle model with Gaussian process noise fit from data. We used the same value function that was trained entirely in the cluttered environment simulation. We also trained a value function with an incorrect wheelbase, $L = 0.5$, to demonstrate the utility of our approach even in the presence of model mismatch.

We generated 20 random environments consisting of a maximum of 8 circular obstacles in an 8 meter by 6 meter space. We discarded environments in which obstacles overlapped or in which no obstacles were blocking the path to

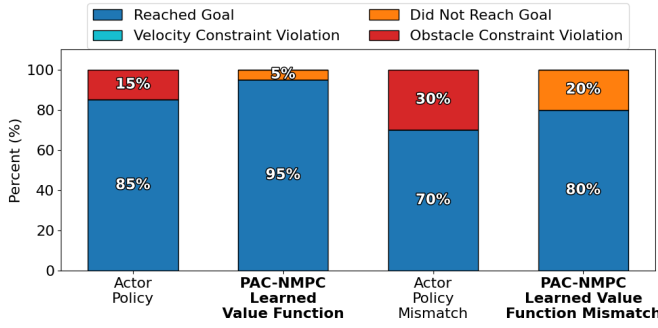


Fig. 10. Hardware environments. Percentage of trials in which the system reached the goal, didn't reach the goal, and collided.

the goal. We placed barrels at these random positions in the motion capture system. In the hardware experiment environments, the Velodyne Puck LITE LiDAR has noise and returns range measurements to the walls, which was not simulated in simulation environments. Thus, these environments are, to some extent, outside of the training distribution.

Our approach outperformed the actor policy and never collided with obstacles (Fig. 10). This indicates that utilizing the RL models inside PAC-NMPC provided better robustness to out-of-distribution environments.

When using the learned value function trained with the incorrect wheelbase, but using the correct wheelbase when sampling PAC-NMPC trajectories, our approach still outperformed the actor policy and never collided with obstacles. Thus, our approach was able to safely utilize an RL model trained in the presence of model mismatch.

VII. DISCUSSION & CONCLUSION

In this paper, we presented an approach for combining an RL-trained value function with sampling-based NMPC to achieve probabilistically safe perception-based navigation using only current sensor measurements. We demonstrated, both in simulation and on hardware, that using the learned value function as a terminal cost and constraint enabled the controller to exhibit long-horizon planning while satisfying collision-avoidance constraints. Further, we showed that our approach was able to generate statistical guarantees of performance and safety in real time, which may improve confidence in the controller's ability to use learned components in safety critical environments. We also demonstrated that our approach can scale to complex, high-dimensional systems and that value functions trained on simple dynamics can aid in the control of more complex systems.

Our approach included several limiting assumptions. It assumed the ability to adequately predict future sensor measurements at the terminal NMPC state as well as an accurate stochastic dynamics model. Additionally, while Monte Carlo dropout was able to quantify epistemic model uncertainty, it would be unable to quantify aleatoric uncertainty (i.e., from sensor noise). Our approach also can be affected by uncalibrated uncertainty estimates. Future work could explore the incorporation of perception uncertainty, as well as online refinement of the learned value function.

REFERENCES

- [1] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "PAMPC: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 1–8. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8593739>
- [2] B. Brito, B. Floor, L. Ferranti, and J. Alonso-Mora, "Model predictive contouring control for collision avoidance in unstructured dynamic environments," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4459–4466, 2019. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8768044>
- [3] A. Polevoy, M. Basescu, L. Scheuer, and J. Moore, "Post-stall navigation with fixed-wing UAVs using onboard vision," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 9696–9702. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9812099>
- [4] A. Polevoy, C. Knuth, K. M. Popek, and K. D. Katyal, "Complex terrain navigation via model error prediction," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 9411–9417. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9811644>
- [5] T. Lew and M. Pavone, "Sampling-based reachability analysis: A random set theory approach with adversarial sampling," in *Conference on robot learning*. PMLR, 2021, pp. 2055–2070.
- [6] J. Yin, Z. Zhang, and P. Tsiotras, "Risk-aware model predictive path integral control using conditional value-at-risk," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 7937–7943.
- [7] A. Polevoy, M. Kobilarov, and J. Moore, "Probably Approximately Correct Nonlinear Model Predictive Control (PAC-NMPC)," *IEEE Robotics and Automation Letters*, pp. 1–8, 2023. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10250934>
- [8] Z. Jian, Z. Yan, X. Lei, Z. Lu, B. Lan, X. Wang, and B. Liang, "Dynamic control barrier function-based model predictive control to safety-critical obstacle-avoidance of mobile robot," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 3679–3685. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10160857>
- [9] C. Wang, X. Chen, C. Li, R. Song, Y. Li, and M. Q.-H. Meng, "Chase and track: Toward safe and smooth trajectory planning for robotic navigation in dynamic environments," *IEEE Transactions on Industrial Electronics*, vol. 70, no. 1, pp. 604–613, 2022. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9709214>
- [10] B. Singh, R. Kumar, and V. P. Singh, "Reinforcement learning in robotic applications: a comprehensive survey," *Artificial Intelligence Review*, vol. 55, no. 2, pp. 945–990, 2022.
- [11] K. Zhu and T. Zhang, "Deep reinforcement learning based mobile robot navigation: A review," *Tsinghua Science and Technology*, vol. 26, no. 5, pp. 674–691, 2021.
- [12] L. Dong, Z. He, C. Song, and C. Sun, "A review of mobile robot motion planning methods: from classical motion planning workflows to reinforcement learning-based architectures," *Journal of Systems Engineering and Electronics*, vol. 34, no. 2, pp. 439–459, 2023.
- [13] P. Mirowski, R. Pascanu, F. Viola, H. Soyer, A. J. Ballard, A. Banino, M. Denil, R. Goroshin, L. Sifre, K. Kavukcuoglu, *et al.*, "Learning to navigate in complex environments," *arXiv preprint arXiv:1611.03673*, 2016.
- [14] Y. Zhu, R. Mottaghi, E. Kolve, J. J. Lim, A. Gupta, L. Fei-Fei, and A. Farhadi, "Target-driven visual navigation in indoor scenes using deep reinforcement learning," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 3357–3364.
- [15] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 6252–6259.
- [16] M. Kobilarov, "Sample complexity bounds for iterative stochastic policy optimization," *Advances in Neural Information Processing Systems*, vol. 28, 2015. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2015/hash/97d98119037c5b8a9663cb21fb8ebf47-Abstract.html
- [17] S. Sharma and M. E. Taylor, "Autonomous waypoint generation strategy for on-line navigation in unknown environments," *environment*, vol. 2, p. 3D, 2012. [Online]. Available: http://www.reflexes.ws/iros2012ws/Paper_12.pdf

- [18] E. Kaufmann, A. Loquercio, R. Ranftl, A. Dosovitskiy, V. Koltun, and D. Scaramuzza, "Deep drone racing: Learning agile flight in dynamic environments," in *Conference on Robot Learning*. PMLR, 2018, pp. 133–145. [Online]. Available: <https://proceedings.mlr.press/v87/kaufmann18a.html>
- [19] C. Greatwood and A. G. Richards, "Reinforcement learning and model predictive control for robust embedded quadrotor guidance and control," *Autonomous Robots*, vol. 43, pp. 1681–1693, 2019. [Online]. Available: <https://link.springer.com/article/10.1007/s10514-019-09829-4>
- [20] S. Bansal, V. Tolani, S. Gupta, J. Malik, and C. Tomlin, "Combining optimal control and learning for visual navigation in novel environments," in *Conference on Robot Learning*. PMLR, 2020, pp. 420–429. [Online]. Available: <https://proceedings.mlr.press/v100/bansal20a>
- [21] B. Brito, M. Everett, J. P. How, and J. Alonso-Mora, "Where to go next: Learning a subgoal recommendation policy for navigation in dynamic environments," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4616–4623, 2021. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9385847>
- [22] K. Bektaş and H. I. Bozma, "Apf-rl: Safe mapless navigation in unknown environments," in *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, 2022, pp. 7299–7305.
- [23] Y. Gao, J. Wu, X. Yang, and Z. Ji, "Efficient hierarchical reinforcement learning for mapless navigation with predictive neighbouring space scoring," *IEEE Transactions on Automation Science and Engineering*, 2023.
- [24] H. Chen and F. Allgöwer, "A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability," *Automatica*, vol. 34, no. 10, pp. 1205–1217, 1998. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0005109898000739>
- [25] M. Zhong, M. Johnson, Y. Tassa, T. Erez, and E. Todorov, "Value function approximation and model predictive control," in *2013 IEEE symposium on adaptive dynamic programming and reinforcement learning (ADPRL)*. IEEE, 2013, pp. 100–107. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/6614995>
- [26] K. Lowrey, A. Rajeswaran, S. Kakade, E. Todorov, and I. Mordatch, "Plan online, learn offline: Efficient learning and exploration via model-based control," *arXiv preprint arXiv:1811.01848*, 2018. [Online]. Available: <https://arxiv.org/abs/1811.01848>
- [27] Z.-W. Hong, J. Pajarinen, and J. Peters, "Model-based lookahead reinforcement learning," *arXiv preprint arXiv:1908.06012*, 2019. [Online]. Available: <https://arxiv.org/abs/1908.06012>
- [28] D. Hoeller, F. Farshidian, and M. Hutter, "Deep value model predictive control," in *Conference on Robot Learning*. PMLR, 2020, pp. 990–1004. [Online]. Available: <https://proceedings.mlr.press/v100/hoeller20a.html>
- [29] M. Bhardwaj, S. Choudhury, and B. Boots, "Blending mpc & value function approximation for efficient reinforcement learning," *arXiv preprint arXiv:2012.05909*, 2020. [Online]. Available: <https://arxiv.org/abs/2012.05909>
- [30] S. Gu, L. Yang, Y. Du, G. Chen, F. Walter, J. Wang, and A. Knoll, "A review of safe reinforcement learning: Methods, theory and applications," *arXiv preprint arXiv:2205.10330*, 2022.
- [31] L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig, "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 5, no. 1, pp. 411–444, 2022.
- [32] P. Geibel and F. Wysotzki, "Risk-sensitive reinforcement learning applied to control under constraints," *Journal of Artificial Intelligence Research*, vol. 24, pp. 81–108, 2005.
- [33] E. Altman, "Constrained markov decision processes with total cost criteria: Lagrangian approach and dual linear program," *Mathematical methods of operations research*, vol. 48, pp. 387–417, 1998.
- [34] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *International conference on machine learning*. PMLR, 2017, pp. 22–31.
- [35] H. Bharadhwaj, A. Kumar, N. Rhinehart, S. Levine, F. Shkurti, and A. Garg, "Conservative safety critics for exploration," *arXiv preprint arXiv:2010.14497*, 2020.
- [36] H. Yu, W. Xu, and H. Zhang, "Towards safe reinforcement learning with a safety editor policy," *Advances in Neural Information Processing Systems*, vol. 35, pp. 2608–2621, 2022.
- [37] A. Anand, K. Seel, V. Gjerum, A. Håkansson, H. Robinson, and A. Saad, "Safe learning for control using control lyapunov functions and control barrier functions: A review," *Procedia Computer Science*, vol. 192, pp. 3987–3997, 2021.
- [38] Y. Chow, O. Nachum, E. Duenez-Guzman, and M. Ghavamzadeh, "A lyapunov-based approach to safe reinforcement learning," *Advances in neural information processing systems*, vol. 31, 2018.
- [39] J. F. Fisac, N. F. Lugovoy, V. Rubies-Royo, S. Ghosh, and C. J. Tomlin, "Bridging hamilton-jacobi safety analysis and reinforcement learning," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8550–8556.
- [40] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 3387–3395.
- [41] Z. Marvi and B. Kiumarsi, "Safe reinforcement learning: A control barrier function optimization approach," *International Journal of Robust and Nonlinear Control*, vol. 31, no. 6, pp. 1923–1940, 2021.
- [42] H. Ma, J. Chen, S. Eben, Z. Lin, Y. Guan, Y. Ren, and S. Zheng, "Model-based constrained reinforcement learning using generalized control barrier function," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 4552–4559.
- [43] T.-Y. Yang, T. Zhang, L. Luu, S. Ha, J. Tan, and W. Yu, "Safe reinforcement learning for legged locomotion," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 2454–2461.
- [44] M. Shekells, G. Garimella, S. Michra, and M. Kobilarov, "Actor-critic pac robust policy search," *ICRA 2019*, 2019. [Online]. Available: <https://par.nsf.gov/servlets/purl/10136847>
- [45] M. Zanon and S. Gros, "Safe reinforcement learning using robust mpc," *IEEE Transactions on Automatic Control*, vol. 66, no. 8, pp. 3638–3652, 2020.
- [46] S. Gros and M. Zanon, "Learning for mpc with stability & safety guarantees," *Automatica*, vol. 146, p. 110598, 2022.
- [47] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, 2020. [Online]. Available: <https://www.andrew.cmu.edu/course/10-703/textbook/BartoSutton.pdf>
- [48] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 31–36. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8202134>
- [49] J. C. de Jesus, V. A. Kich, A. H. Kolling, R. B. Grando, M. A. d. S. L. Cuadros, and D. F. T. Gamarra, "Soft actor-critic for navigation of mobile robots," *Journal of Intelligent & Robotic Systems*, vol. 102, no. 2, p. 31, 2021. [Online]. Available: <https://link.springer.com/article/10.1007/s10846-021-01367-5>
- [50] H. Beomsoo, A. A. Ravankar, and T. Emaru, "Mobile robot navigation based on deep reinforcement learning with 2d-lidar sensor using stochastic approach," in *2021 IEEE International Conference on Intelligence and Safety for Robotics (ISR)*. IEEE, 2021, pp. 417–422. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9419565>
- [51] S. Huang, R. F. J. Dossa, C. Ye, J. Braga, D. Chakraborty, K. Mehta, and J. G. Araújo, "Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms," *Journal of Machine Learning Research*, vol. 23, no. 274, pp. 1–18, 2022. [Online]. Available: <http://jmlr.org/papers/v23/21-1342.html>
- [52] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596. [Online]. Available: <https://proceedings.mlr.press/v80/fujimoto18a.html>
- [53] Y. Gal and Z. Ghahramani, "Dropout as a bayesian approximation: Representing model uncertainty in deep learning," in *international conference on machine learning*. PMLR, 2016, pp. 1050–1059. [Online]. Available: <https://proceedings.mlr.press/v48/gal16.html?trk=public.post.comment-text>
- [54] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive driving with model predictive path integral control," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1433–1440.
- [55] M. Basescu and J. Moore, "Direct nmpe for post-stall motion planning with fixed-wing uavs," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 9592–9598.